

Project_2

March 1, 2026

Comparative Analysis of KNN and Naive Bayes for Income Level Classification

Author: Gerardo Parra Diaz

Date: 3/1/2026

Course: DAT-402

Introduction

The Adult Income dataset is a collection of 1994 U.S. Census records used to predict whether an individual earns more than \$50,000 per year. It consists of 14 demographic features, including a mix of numerical data like age and categorical data like education or occupation.

The primary objective of this project is to develop and evaluate machine learning models capability of predicting whether an individual's annual income exceeds \$50,000 based on 1994 US Census Bureau data. This is a binary classification problem involving a mix of continuous (numerical) and categorical (string-based) features.

The I got the dataset from the UC Irvine Machine Learning Repository website(<https://archive.ics.uci.edu/dataset/2/adult>)

Research Question: To what extent can K-Nearest Neighbors (KNN) and Naive Bayes classifiers accurately predict an individual's income level (above or below \$50,000) using 1994 Census data, and how do their underlying mathematical assumptions handle high-dimensional, skewed feature sets?

```
[1]: import pandas as pd
import numpy as np

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import classification_report
```

Importing Dataset

```
[2]: # Load it directly
df = pd.read_csv("adult.data")

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32560 entries, 0 to 32559
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   39                    32560 non-null  int64
 1   State-gov            32560 non-null  object
 2   77516                32560 non-null  int64
 3   Bachelors            32560 non-null  object
 4   13                   32560 non-null  int64
 5   Never-married        32560 non-null  object
 6   Adm-clerical         32560 non-null  object
 7   Not-in-family        32560 non-null  object
 8   White                32560 non-null  object
 9   Male                 32560 non-null  object
10   2174                 32560 non-null  int64
11   0                    32560 non-null  int64
12   40                   32560 non-null  int64
13   United-States        32560 non-null  object
14   <=50K                32560 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

Data Wrangling

Using the adults.names file that came with the dataset zip file, descriptions were given for each column, which is what I used to rename the column names to the appropriate variable.

```
[3]: #Create column names since file does not have a header
col_names = ["age", "workclass", "fnlwgt", "education", "education-num",
             "marital-status", "occupation", "relationship", "race", "sex",
             "capital-gain", "capital-loss", "hours-per-week", "native-country",
             ↪ "income"]

df = pd.read_csv("adult.data", names=col_names)

df.head()
```

```
[3]:   age   workclass  fnlwgt  education  education-num \
0   39   State-gov   77516   Bachelors             13
1   50  Self-emp-not-inc  83311   Bachelors             13
2   38    Private  215646    HS-grad              9
```

3	53	Private	234721	11th	7
4	28	Private	338409	Bachelors	13

	marital-status	occupation	relationship	race	sex \
0	Never-married	Adm-clerical	Not-in-family	White	Male
1	Married-civ-spouse	Exec-managerial	Husband	White	Male
2	Divorced	Handlers-cleaners	Not-in-family	White	Male
3	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male
4	Married-civ-spouse	Prof-specialty	Wife	Black	Female

	capital-gain	capital-loss	hours-per-week	native-country	income
0	2174	0	40	United-States	<=50K
1	0	0	13	United-States	<=50K
2	0	0	40	United-States	<=50K
3	0	0	40	United-States	<=50K
4	0	0	40	Cuba	<=50K

Researching the dataset before importing it into Jupyter Notebook, I found that this census bureau dataset is notorious for having whitespaces in some of the entries and '?' for na values. That will cause a problem for some of the code later so I will first see if there are any whitespaces in the dataset.

```
[4]: #this prints the first value of 'workclass' inside brackets to reveal spaces
print(f"{df['workclass'].iloc[0]}")
```

' State-gov'

Since the output was ' state-gove' and not 'state-gove' there is whitespace that Pandas is trimming for readability. I will strip these whitespaces.

```
[5]: df = df.apply(lambda x: x.str.strip() if x.dtype == "object" else x)

print(f"{df['workclass'].iloc[0]}")
```

'State-gov'

Now for the na values, this dataset uses question marks to indicate if an entry was not available.

```
[6]: df.isin(['?']).sum()
```

```
[6]: age                0
workclass             1836
fnlwgt                0
education              0
education-num         0
marital-status        0
occupation            1843
relationship          0
race                  0
sex                   0
```

```
capital-gain      0
capital-loss      0
hours-per-week    0
native-country    583
income            0
dtype: int64
```

As we can see 3 columns contain hundreds or thousands of these question mark entries. We will drop them in the same manner as we would NA values but first we will convert these entries as numpy Na objects.

```
[7]: df.replace('?', np.nan, inplace=True)
df.dropna(inplace=True)

print(f"Rows after dropping: {len(df)}")
```

Rows after dropping: 30162

Since the dataset started with 32561 rows we know we dropped all right rows since we only have 30162 rows left.

Now we will encode the target variable, **income**. Right now it contains strings and ">50k". In order to properly use Naive Bayes and KNN algorithms we need to convert this column into a numeric type variable.

```
[8]: le = LabelEncoder()
df['income'] = le.fit_transform(df['income'])

df_final = pd.get_dummies(df, drop_first=True, dtype=int)

df_final.head()
```

```
[8]:
```

	age	fnlwgt	education-num	capital-gain	capital-loss	hours-per-week	\
0	39	77516	13	2174	0	40	
1	50	83311	13	0	0	13	
2	38	215646	9	0	0	40	
3	53	234721	7	0	0	40	
4	28	338409	13	0	0	40	

	income	workclass_Local-gov	workclass_Private	workclass_Self-emp-inc	\
0	0	0	0	0	
1	0	0	0	0	
2	0	0	1	0	
3	0	0	1	0	
4	0	0	1	0	

	...	native-country_Portugal	native-country_Puerto-Rico	\
0	...	0	0	
1	...	0	0	

2	...	0	0
3	...	0	0
4	...	0	0

	native-country_Scotland	native-country_South	native-country_Taiwan \
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

	native-country_Thailand	native-country_Trinidad&Tobago \
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

	native-country_United-States	native-country_Vietnam \
0	1	0
1	1	0
2	1	0
3	1	0
4	0	0

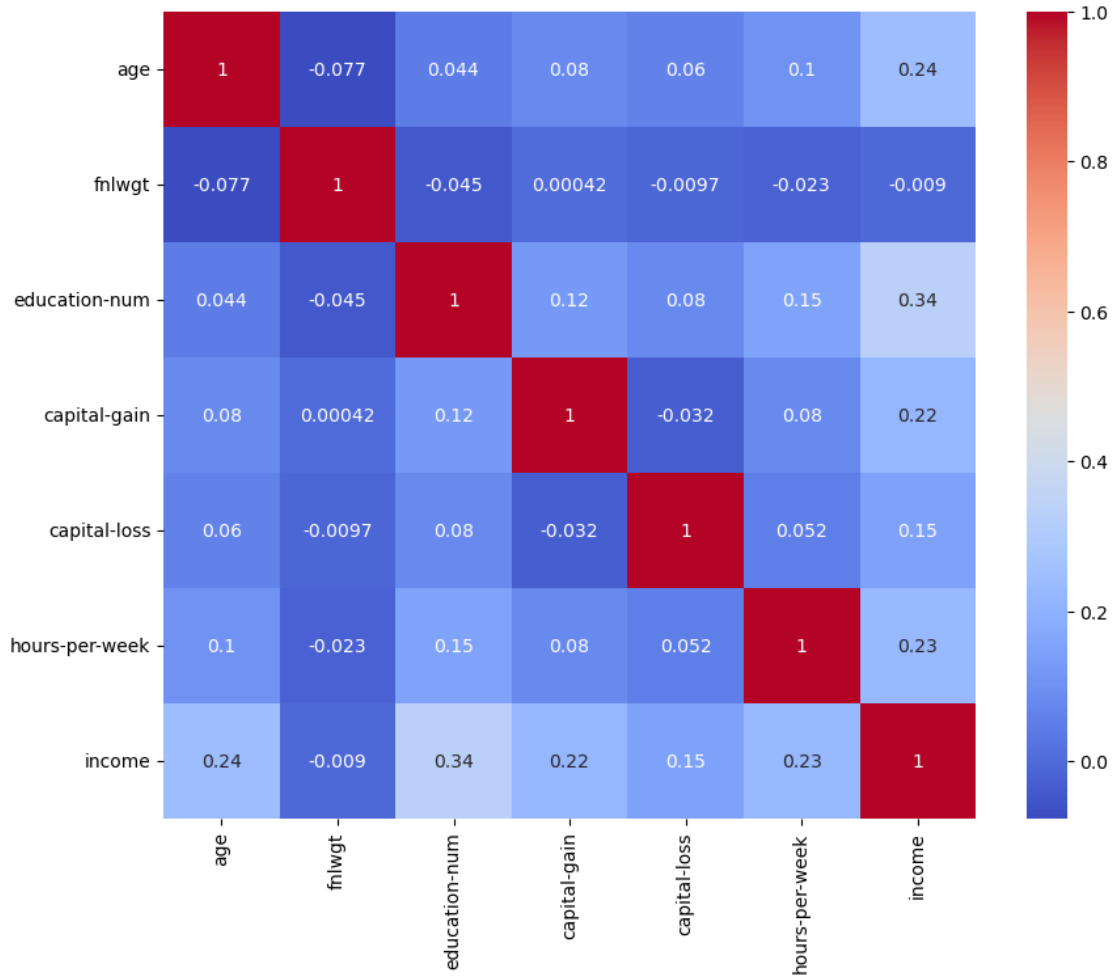
	native-country_Yugoslavia
0	0
1	0
2	0
3	0
4	0

[5 rows x 97 columns]

Exploratory Data Analysis

To understand the relationships within our data, we generated a Pearson correlation heatmap focusing on the numerical features and our target variable, **income**.

```
[9]: plt.figure(figsize=(10, 8))
      sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
      plt.show()
```



The Naive Bayes classifier operates on the Strong Independence Assumption, meaning it assumes that the presence of one feature is unrelated to the presence of any other feature. Our heatmap revealed that several features (like education and occupation-related metrics) are indeed correlated.

Because $r = 0.34$ between education and income, the independence assumption is technically violated. This “feature dependency” is a primary reason why we expect the K-Nearest Neighbors (KNN) model—which does not rely on independence—to potentially outperform the Gaussian Naive Bayes model.

Scaling & Train/Test Split

The features in this dataset exist on vastly different scales. For instance, age typically ranges from 17 to 90, while capital-gain can range from 0 to 99,999.

Because the K-Nearest Neighbors (KNN) algorithm relies on calculating the Euclidean Distance between data points, features with larger numerical ranges would disproportionately dominate the model’s decision-making.

To solve this, we applied StandardScaler to transform the data so that each feature has a mean (μ)

of 0 and a standard deviation (σ) of 1. This ensures that every feature—from education level to hours worked—contributes equally to the distance calculation.

```
[10]: # 1. Separate features and target
X = df_final.drop('income', axis=1)
y = df_final['income']

# 2. Create the 4 variables: X_train, X_test, y_train, y_test
# we use 80% for training and 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↪random_state=42)
```

```
[11]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

To accurately measure how well our models generalize to “new” data, we partitioned the dataset into a 80/20 train test split to fit the models

```
[12]: #Training KNN
# Start with k=5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
knn_preds = knn.predict(X_test_scaled)
```

```
[13]: #Training Naive Bayes
nb = GaussianNB()
nb.fit(X_train_scaled, y_train)
nb_preds = nb.predict(X_test_scaled)
```

In this section, we evaluate the predictive accuracy of our classifiers and investigate the “performance gap” between KNN’s robust distance-based logic and Naive Bayes’ sensitivity to data distribution.

```
[14]: # KNN Predictions
knn_preds = knn.predict(X_test_scaled)

# Naive Bayes Predictions
nb_preds = nb.predict(X_test_scaled)

print(f"KNN Accuracy: {accuracy_score(y_test, knn_preds):.4f}")
print(f"Naive Bayes Accuracy: {accuracy_score(y_test, nb_preds):.4f}")
```

KNN Accuracy: 0.8132

Naive Bayes Accuracy: 0.4502

While KNN achieved a strong 81.32% accuracy, the Gaussian Naive Bayes model significantly underperformed at 45.02%

To move beyond simple accuracy scores, we utilize Confusion Matrices and Classification Reports to visualize where our models are making successful predictions and where they are failing.

```
[15]: confusion_matrix(y_test, knn_preds)
```

```
[15]: array([[4048,  455],
        [ 672,  858]])
```

```
[16]: print("--- KNN Report ---")
print(classification_report(y_test, knn_preds))

print("\n--- Naive Bayes Report ---")
print(classification_report(y_test, nb_preds))
```

--- KNN Report ---

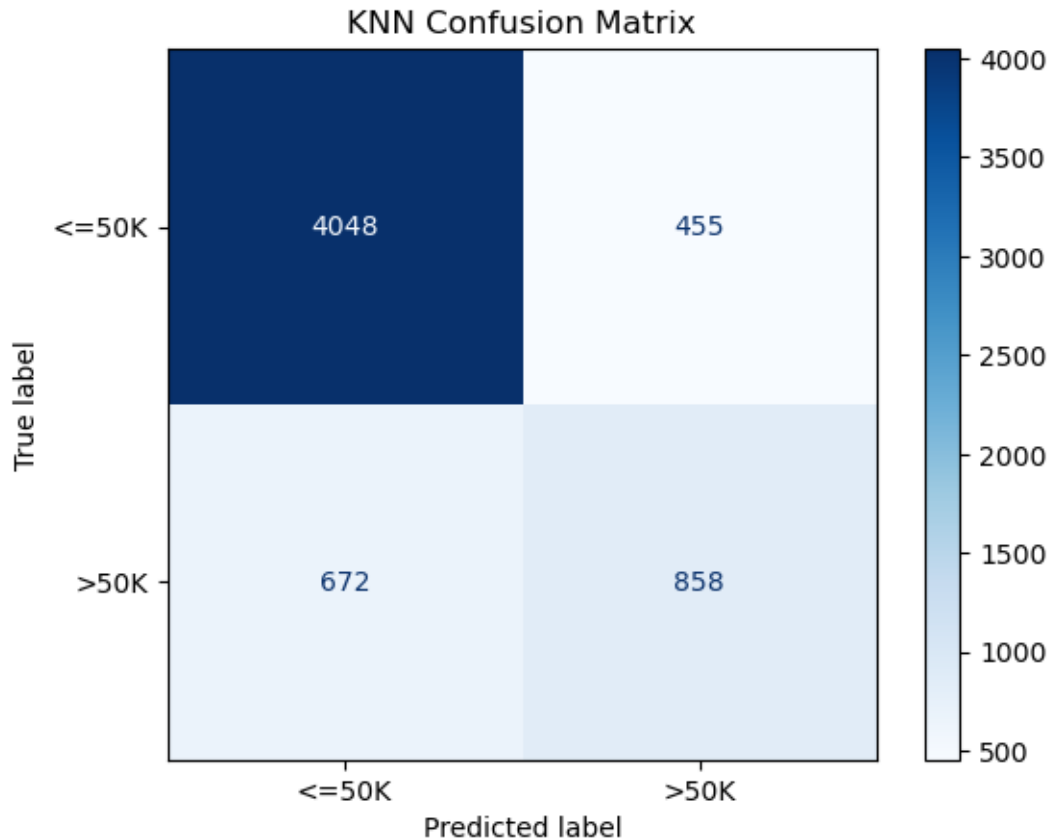
	precision	recall	f1-score	support
0	0.86	0.90	0.88	4503
1	0.65	0.56	0.60	1530
accuracy			0.81	6033
macro avg	0.76	0.73	0.74	6033
weighted avg	0.81	0.81	0.81	6033

--- Naive Bayes Report ---

	precision	recall	f1-score	support
0	0.95	0.28	0.43	4503
1	0.31	0.96	0.47	1530
accuracy			0.45	6033
macro avg	0.63	0.62	0.45	6033
weighted avg	0.79	0.45	0.44	6033

```
[17]: # 1. Generate the raw matrix
cm = confusion_matrix(y_test, knn_preds)

# 2. Plot it
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['<=50K',
↪ '>50K'])
disp.plot(cmap='Blues')
plt.title('KNN Confusion Matrix')
plt.show()
```

The KNN model achieved a balanced performance with an accuracy of 81.32%. By examining its Confusion Matrix, we can see it effectively identifies the majority class (Low Income) while maintaining a respectable “Recall” for the minority class (High Income). Because KNN is distance-based, it is not “confused” by the non-normal distribution of features like capital gains.

The Classification Report for Gaussian NB reveals a critical failure: the model is performing worse than a random guess. In our dataset, most people have \$0 for capital-gain, but a few have 99,999.

Gaussian NB attempts to fit these outliers into a “Bell Curve” (Normal Distribution), which stretches the mathematical probability so thin that the model loses its predictive power.

This proves that our features do not meet the Gaussian assumption, prompting us to test the Bernoulli Naive Bayes variant which is specifically designed for the binary (0/1) dummy variables we created during preprocessing.

```
[18]: df['capital-gain'].describe()
```

```
[18]: count    30162.000000
      mean      1092.007858
      std       7406.346497
      min         0.000000
      25%         0.000000
```

```
50%          0.000000
75%          0.000000
max          99999.000000
Name: capital-gain, dtype: float64
```

```
[19]: print(y_train.value_counts())
```

```
income
0    18151
1     5978
Name: count, dtype: int64
```

```
[20]: from sklearn.naive_bayes import BernoulliNB
      bnb = BernoulliNB()
      bnb.fit(X_train_scaled, y_train)
      print(f"Bernoulli NB Accuracy: {bnb.score(X_test_scaled, y_test):.4f}")
```

```
Bernoulli NB Accuracy: 0.8125
```

Conclusion

The final results of this project demonstrate that K-Nearest Neighbors (KNN) and Bernoulli Naive Bayes are equally effective at predicting income levels, both achieving an accuracy of approximately 81.3%.

The most significant finding of this research was the failure of the Gaussian Naive Bayes model (45.02%), which highlighted the importance of matching an algorithm's mathematical assumptions to the data's distribution.

Improve predictive power, implementing ensemble methods like Random Forest would likely be more robust, as these models naturally handle the skewed distributions that challenged our Gaussian Naive Bayes approach. Overall, these refinements would transition the project from a baseline comparison to a highly tuned, production-ready classification system.